INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 5 LECTURE 1

INTERRUPTS & EXCEPTIONS

WISH YOU ALL THE BEST

INTERRUPTS

- An interrupt is usually defined as an event that alters the sequence of instructions executed by a processor.
- Interrupts are often divided into synchronous and asynchronous interrupts:

  • Synchronous interrupts

  are produced by the CPU control unit while executing instructions and are called synchronous because the control unit issues them only after terminating the execution of an instruction.

  • Asynchronous interrupts

  are generated by other hardware devices at arbitrary times with respect to the CPU clock signals.

- Intel 80x86 microprocessor manuals designate synchronous and asynchronous interrupts as exceptions and interrupts, respectively.

INTERRUPTS

- **Interrupts are issued by interval timers and I/O devices; for instance, the arrival of a keystroke from a user sets off an interrupt.**
- **Exceptions, on the other hand, are caused either by programming errors or by anomalous conditions that must be handled by the kernel.**
- **In the first step, the kernel handles the exception by delivering to the current process one of the signals familiar to every Unix programmer.**
- **In the second step, the kernel performs all the steps needed to recover from the anomalous condition, such as a page fault or a request (via an int instruction) for a kernel service.**

INTERRUPTS

Role of Interrupt Signals

- **Interrupt signals provide a way to divert the processor to code outside the normal flow of control.**
- **When an interrupt signal arrives, the CPU must stop what it's currently doing and switch to a new activity**
- **It does this by saving the current value of the program counter (i.e., the content of the eip and cs registers) in the Kernel Mode stack and by placing an address related to the interrupt type into the program counter.**

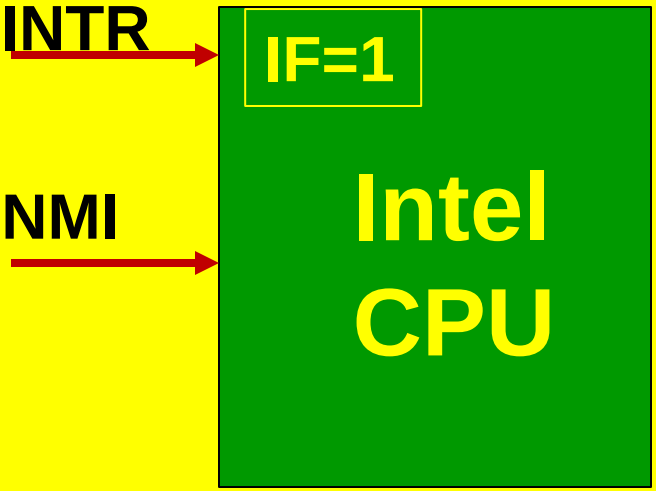
INTERRUPTS

Role of Interrupt Signals

- There is a key difference between interrupt handling and process switching: the code executed by an interrupt or by an exception handler is not a process.
- Rather, it is a kernel control path that runs on behalf of the same process that was running when the interrupt occurred.
- As a kernel control path, the interrupt handler is lighter than a process (it has less context and requires less time to set up or tear down).

INTERRUPTS

- INTERRUPT PINS ON INTEL 80X86
  - TWO TYPES OF HARDWARE INTERRUPTS
    - Maskable interrupts
    - Nonmaskable interrupts



INTERRUPTS

- **Maskable interrupts**

- Sent to the INTR pin of the microprocessor.
- They can be disabled by clearing the IF flag of the eflags register.
- All IRQs issued by I/O devices give rise to maskable interrupts.

- **Nonmaskable interrupts**

- Sent to the NMI (Nonmaskable Interrupts) pin of the microprocessor.
- They are not disabled by clearing the IF flag. Only a few critical events, such as hardware failures, give rise to nonmaskable interrupts.

EXCEPTIONS

- TWO TYPES OF EXCEPTIONS
- Processor-detected exceptions
- Programmed exceptions

EXCEPTIONS

- **Processor-detected exceptions**

- Generated when the CPU detects an anomalous condition while executing an instruction.
- These are further **divided into three groups, depending on the value of the eip register** that is saved on the Kernel Mode stack when the CPU control unit raises the exception:
  - **Faults**
  - **Traps**
  - **Aborts**

EXCEPTIONS

- **Processor-detected exceptions**

  - **Faults**

  - The saved value of eip is the address of the instruction that caused the fault, and hence that instruction can be resumed when the exception handler terminates.

  - Resuming the same instruction is necessary whenever the handler is able to correct the anomalous condition that caused the exception.

EXCEPTIONS

- **Processor-detected exceptions**
- **Traps**
  - The saved value of eip is the address of the instruction that should be executed after the one that caused the trap.
  - A trap is triggered only when there is no need to re-execute the instruction that terminated.
  - The main use of traps is for debugging purposes: the role of the interrupt signal in this case is to notify the debugger that a specific instruction has been executed (for instance, a breakpoint has been reached within a program).
  - Once the user has examined the data provided by the debugger, user may ask that execution of the debugged program resume starting from the next instruction.

EXCEPTIONS

- **Processor-detected exceptions**
- **Aborts**
  - A serious error occurred; the control unit is in trouble, and it may be unable to store a meaningful value in the eip register.
  - Aborts are caused by hardware failures or by invalid values in system tables.
  - The interrupt signal sent by the control unit is an emergency signal used to switch control to the corresponding abort exception handler.
  - This handler has no choice but to force the affected process to terminate.

EXCEPTIONS

Programmed exceptions

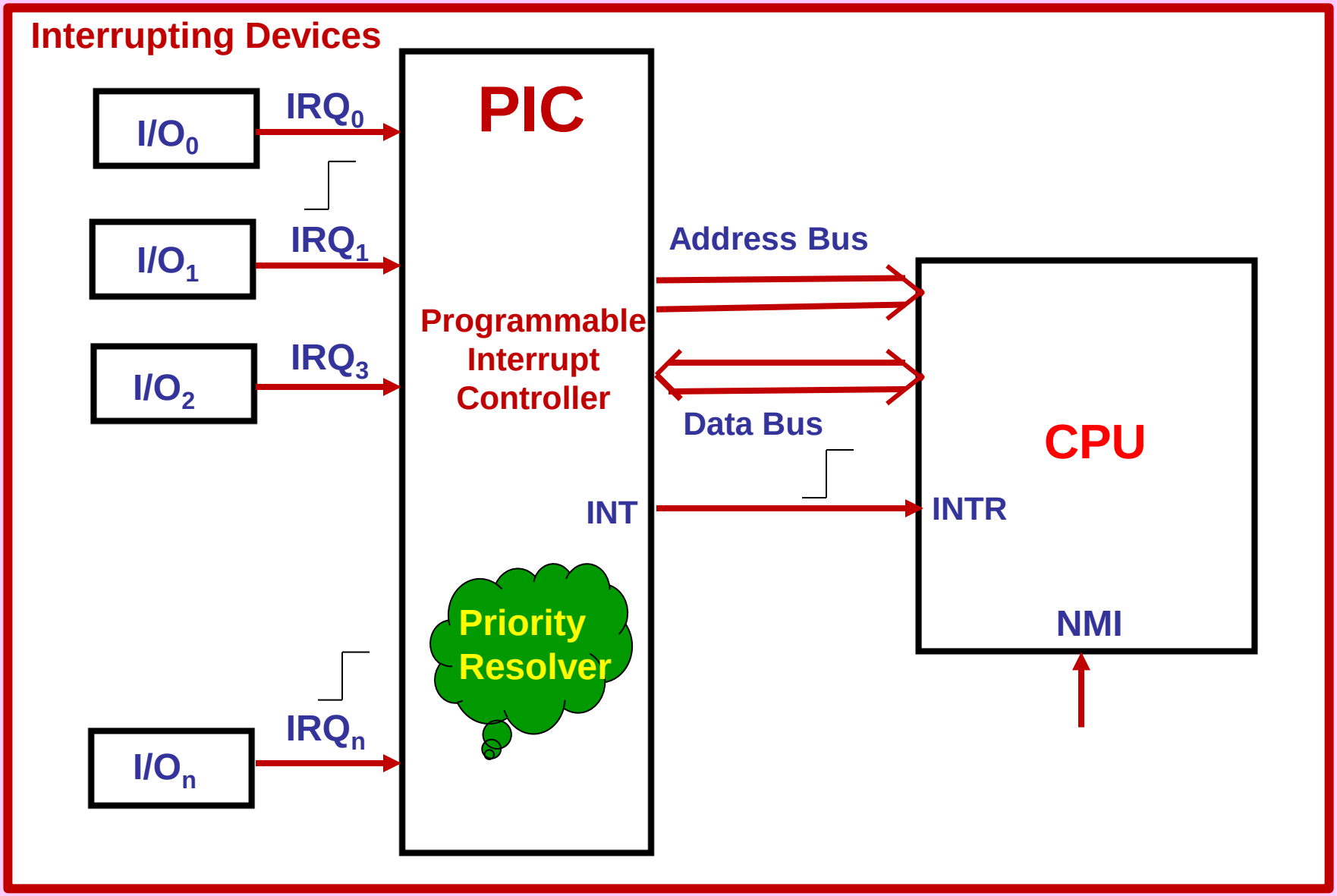
- Occur at the request of the programmer.
- They are triggered by **int or int 3, 4, 5 instructions**;
- **The int 3 (Breakpoint), int 4 (check for overflow) and int 5 bound (check on address bound) instructions** also give rise to a programmed exception when the condition they are checking is not true.
- Programmed exceptions are handled by the control unit as **traps**; they are often called **software interrupts.**
- **Such exceptions have two common uses: to implement system calls, and to notify a debugger of a specific event**

Interrupt and Exception Vectors

- Each interrupt or exception is identified by a number ranging from **0 to 255**; Intel calls this **8-bit unsigned number a vector.**
- The vectors of nonmaskable interrupts and exceptions are fixed, while those of maskable interrupts can be altered by programming the Interrupt Controller
- Linux uses the following vectors**:**
  - **Vectors ranging from 0 to 31** correspond to **exceptions and nonmaskable interrupts.**
  - **Vectors ranging from 32 to 47** are assigned to **maskable interrupts**, that is, to interrupts caused by IRQs.
  - The remaining vectors ranging from 48 to 255 may be used to identify **software interrupts. Linux uses only one of them, namely the 128 or 0x80 vector, which it uses to implement system calls**. **When an int 0x80 Assembly instruction is executed by a process in User Mode, the CPU switches into Kernel Mode and starts executing the system_call( ) kernel function**

INTERRUPTS

IRQs and Interrupts



IRQs and Interrupts

- **Each hardware device controller capable of issuing interrupt requests has an output line designated as an IRQ (Interrupt ReQuest).**
- **All existing IRQ lines are connected to the input pins of a hardware circuit called the Interrupt Controller, which performs the following actions:**

1. **Monitors the IRQ lines, checking for raised signals.**
2. **If a raised signal occurs on an IRQ line:**
   - **a. Converts the raised signal received into a corresponding vector.**
   - **b. Stores the vector in an Interrupt Controller I/O port, thus allowing the CPU to read it via the data bus.**
   - **c. Sends a raised signal to the processor INTR pin—that is, issues an interrupt.**
   - **d. Waits until the CPU acknowledges the interrupt signal by writing into one of the Programmable Interrupt Controllers (PIC) I/O ports; when this occurs, clears the INTR line.**
3. **Goes back to step 1.**

- **The IRQ lines are sequentially numbered starting from 0; thus, the first IRQ line is usually denoted as IRQ0. Intel's default vector associated with IRQn is n+32; the mapping between IRQs and vectors can be modified by issuing suitable I/O instructions to the Interrupt Controller ports.**

SOC 3010
OPERATING SYSTEM

END OF WEEK 5 LECTURE 1

INTERRUPTS & EXCEPTIONS

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG